Module 12 Lab Session 1: .NET Unit, Mocking, and Integration Testing

Session 1 of 2

Exercises Exercises **1**, **2**, and **3**

Integrity **Tier 1** xUnit `[Fact]`/`[Theory]` boundary testing, NSubstitute mocks,
Lab tier `WebApplicationFactory` API contract assertions

Story

A student double-clicked **Submit Enrollment** and created duplicate rows. No test failed because no automated test existed. You will build the unit and integration test safety net that catches bugs like this in C# domain logic and the HTTP pipeline before they reach production.

Exercise 1: The Pure Logic Test (xUnit)

The TMS stores per-enrollment grades as a `decimal` (M5's `Enrollment.Grade`) and per-assessment maximums as `decimal MaxScore` (M5's `Assessment`). Letter grades do not exist yet, you will add a small `GradingService` to `TmsApi.Application` that maps a `(score, maxScore)` pair to a `GradeLevel`. Unit tests pin those rules, so future refactoring cannot break grading logic silently. This is a deliberate TMS project push during M12: by the end of Exercise 1 you ship one new service plus its tests.

Step 1: Create the Test Project

Open a terminal in your TMS API solution directory:

```
dotnet new xunit -n TmsApi.Tests
dotnet sln add TmsApi.Tests
cd TmsApi.Tests
dotnet add reference ../TmsApi.Application/TmsApi.Application.csproj
```

(Add `TmsApi.Application` as a project reference. `TmsApi.Application` is where `IEnrollmentService` already lives per the M7 Session 0 refactor; the new `GradingService` belongs here too.)

Step 2: Add GradingService to the TMS project

Create `TmsApi.Application/Grading/GradeLevel.cs`:

```
namespace TmsApi.Application.Grading;

public enum GradeLevel
{
    Distinction,
    Pass,
    Fail,
    Invalid,
}
```

Create `TmsApi.Application/Grading/GradingService.cs`:

```
namespace TmsApi.Application.Grading;

public class GradingService
{
    public const decimal DistinctionThreshold = 70m;
    public const decimal PassThreshold = 50m;

    // Pure mapping: one score against one maximum.
    // Uses M5's Assessment.MaxScore and the decimal part of
//Enrollment.Grade.
    public GradeLevel CalculateLetterGrade(decimal score, decimal maxScore)
    {
        if (maxScore <= 0m || score < 0m || score > maxScore)
            return GradeLevel.Invalid;

        var pct = score / maxScore * 100m;
        return pct >= DistinctionThreshold ? GradeLevel.Distinction
             : pct >= PassThreshold       ? GradeLevel.Pass
             :                              GradeLevel.Fail;
    }
```

```
    // Single-decimal path: maps an Enrollment.Grade percentage to a
GradeLevel.
    // Enrollment.Grade is nullable per the M5 entity; null => Invalid.
    public GradeLevel CalculateFromEnrollmentGrade(decimal?
enrollmentGradePercent)
    {
        if (enrollmentGradePercent is null) return GradeLevel.Invalid;
        return CalculateLetterGrade(enrollmentGradePercent.Value, maxScore:
100m);
    }
}
```

What these exercises: `CalculateLetterGrade` consumes the existing `Assessment` shape from M5 (`decimal MaxScore`, `decimal Weight`). `CalculateFromEnrollmentGrade` consumes the existing `Enrollment.Grade` decimal. No new domain model, only a new application layer service that interprets already stored data.

Step 3: Write the First Fact Test

Create `GradingServiceTests.cs` in `TmsApi.Tests`:

```
using TmsApi.Application.Grading;

namespace TmsApi.Tests;

public class GradingServiceTests
{
    [Fact]
    public void CalculateLetterGrade_HighScore_ReturnsDistinction()
    {
        // Arrange
        var service = new GradingService();
        // Act
        var result = service.CalculateLetterGrade(score: 85m, maxScore: 100m);
        // Assert
        Assert.Equal(GradeLevel.Distinction, result);
    }
}
```

Step 4: Add Parameterized Theory Tests

Add boundary cases to `GradingServiceTests.cs`:

```
[Theory]
[InlineData(0,   100, GradeLevel.Fail)]        // Boundary: zero score
[InlineData(70,  100, GradeLevel.Distinction)] // Boundary: at distinction
//threshold
[InlineData(50,  100, GradeLevel.Pass)]       // Boundary: at pass threshold
[InlineData(-1,  100, GradeLevel.Invalid)]    // Boundary: negative score
[InlineData(101, 100, GradeLevel.Invalid)]    // Boundary: score exceeds max
[InlineData(50,  0,   GradeLevel.Invalid)]    // Boundary: zero max score
//(undefined percentage)
public void CalculateLetterGrade_VariousInputs_ReturnsExpectedLevel(
    decimal score, decimal maxScore, GradeLevel expected)
{
    var service = new GradingService();
    var result = service.CalculateLetterGrade(score, maxScore);
    Assert.Equal(expected, result);
}
```

Step 5: Run the Tests

```
dotnet test
```

Expected result: All tests pass. If the grading logic has bugs, one or more tests fail. Fix the production code, never dilute the test assertions.

Exercise 2: Mocking External Boundaries (NSubstitute)

Context: Your M7 handler `EnrollStudentHandler` depends on the application-layer interface `IEnrollmentService`. To test the handler's behaviour (the place where route logic, validation, and authorization live) without hitting the database, you mock `IEnrollmentService` and feed the handler the fake.

Step 1: Install NSubstitute

In your `TmsApi.Tests` directory:

```
dotnet add package NSubstitute
```

Step 2: Write the Mock Test

Create `EnrollStudentHandlerTests.cs`:

```
using NSubstitute;
using TmsApi.Application.Interfaces;
using TmsApi.Application.Enrollments.Commands;
using TmsApi.Application.Common;
using TmsApi.Domain.Entities;

namespace TmsApi.Tests;

public class EnrollStudentHandlerTests
{
    [Fact]
    public async Task Handle_WhenAlreadyEnrolled_ReturnsDuplicateError()
    {
  // Arrange: create a mock IEnrollmentService (Application-layer interface)
        var enrollmentService = Substitute.For<IEnrollmentService>();
        var courseService = Substitute.For<ICourseService>();

        enrollmentService
            .ExistsAsync(99, "CS-401", Arg.Any<CancellationToken>())
            .Returns(Task.FromResult(true));

        // Course lookup runs first in the handler; return any non-null
// course so the duplicate check is the branch under test.
        var course = new Course
        {
            Id = 1,
            Code = "CS-401",
            Title = "Advanced Web Dev",
            MaxCapacity = 30,
            Enrollments = new List<Enrollment>(),
        };
        courseService
            .GetByCodeAsync("CS-401", Arg.Any<CancellationToken>())
            .Returns(Task.FromResult<Course?>(course));

  var handler = new EnrollStudentHandler(enrollmentService, courseService);
  var command=new EnrollStudentCommand(StudentId: 99, CourseCode: "CS-401");
```

```
        // Act
        var result = await handler.Handle(command, CancellationToken.None);

// Assert: handler surfaces the duplicate without touching the database.
// Assert on the machine-readable Code (the contract) plus full record
// equality, NOT on the human-readable Message,see M7 sealed-record pattern.
        Assert.False(result.IsSuccess);
        Assert.Equal("already_enrolled", result.Error.Code);
        Assert.Equal(EnrollmentError.AlreadyEnrolled(99, "CS-401"),
result.Error);

        // The duplicate branch never writes — prove it.
        await enrollmentService
            .DidNotReceive()
            .AddAsync(Arg.Any<Enrollment>(), Arg.Any<CancellationToken>());
    }

    [Fact]
    public async Task Handle_WhenCourseFull_ReturnsCapacityError()
    {
   // Arrange: course is at capacity (Enrollments.Count >= MaxCapacity).
 // M7's handler checks capacity against the course object, not via service
// calls.
        var enrollmentService = Substitute.For<IEnrollmentService>();
        var courseService = Substitute.For<ICourseService>();

        var course = new Course
        {
            Id = 1,
            Code = "CS-401",
            Title = "Advanced Web Dev",
            MaxCapacity = 35,
            Enrollments = Enumerable.Range(1, 35)
                .Select(i => new Enrollment { Id = i, CourseId = 1, Status =
"Pending"})
                .ToList()};
        courseService
            .GetByCodeAsync("CS-401", Arg.Any<CancellationToken>())
            .Returns(Task.FromResult<Course?>(course));
```

```
var handler= new EnrollStudentHandler(enrollmentService, courseService);
var command = new EnrollStudentCommand(StudentId:100, CourseCode: "CS-401");

        // Act
        var result = await handler.Handle(command, CancellationToken.None);

        // Assert: typed error matches the M7 sealed-record factory
        Assert.False(result.IsSuccess);
        Assert.Equal("course_full", result.Error.Code);
        Assert.Equal(EnrollmentError.CourseFull("Advanced Web Dev", 35),
result.Error);

        await enrollmentService
            .DidNotReceive()
            .AddAsync(Arg.Any<Enrollment>(), Arg.Any<CancellationToken>());
    }

    [Fact]
    public async Task Handle_SuccessfulPath_AddsEnrollmentOnce()
    {
  // Arrange: course has room, student is not already enrolled; expect one
// AddAsync call.
        var enrollmentService = Substitute.For<IEnrollmentService>();
        var courseService = Substitute.For<ICourseService>();

        var course = new Course
        {
            Id = 1,
            Code = "CS-401",
            Title = "Advanced Web Dev",
            MaxCapacity = 35,
            Enrollments = Enumerable.Range(1, 20)
                .Select(i => new Enrollment { Id = i, CourseId = 1, Status=
"Pending", })
                .ToList(),
        };
        courseService
            .GetByCodeAsync("CS-401", Arg.Any<CancellationToken>())
            .Returns(Task.FromResult<Course?>(course));
        enrollmentService
            .ExistsAsync(100, "CS-401", Arg.Any<CancellationToken>())
            .Returns(Task.FromResult(false));
```

```
 var handler = new EnrollStudentHandler(enrollmentService, courseService);
var command =new EnrollStudentCommand(StudentId: 100, CourseCode: "CS-401");

        // Act
        var result = await handler.Handle(command, CancellationToken.None);

      // Assert: handler produced a typed success payload with the right IDs
        Assert.True(result.IsSuccess);
        Assert.Equal(100, result.Value.StudentId);
        Assert.Equal("CS-401", result.Value.CourseCode);

// The interaction: AddAsync called exactly once with a row that points at
// the student and course
        await enrollmentService
            .Received(1)
            .AddAsync(
                Arg.Is<Enrollment>(e => e.StudentId == 100 && e.CourseId ==
1),
                Arg.Any<CancellationToken>());
    }
}
```

Why this version: `EnrollStudentHandler` (M7's MediatR handler) takes `IEnrollmentService` and `ICourseService` , both Application-layer interfaces. Mocking both keeps the handler testable without spinning up an EF in-memory provider. The handler's real branches map cleanly to three tests: duplicate enrollment, course-full, and successful write each test asserts the typed `Result` plus a `Received()` / `DidNotReceive()` interaction so a future refactor cannot silently break the boundary.

Exercise 3: Integration Testing (WebApplicationFactory)

Unit tests verify C# logic in isolation. Integration tests verify that the entire ASP.NET Core pipeline, routing, middleware, controllers, model binding, and JSON serialization, operates correctly together under HTTP requests.

Step 1: Install Integration Testing Packages

In your `TmsApi.Tests` directory, install `Microsoft.AspNetCore.Mvc.Testing`, the EF Core in-memory provider, and reference the API project:

```
dotnet add package Microsoft.AspNetCore.Mvc.Testing
dotnet add package Microsoft.EntityFrameworkCore.InMemory
dotnet add reference ../TmsApi.Api/TmsApi.Api.csproj
```

> **Why `InMemory`?** `WebApplicationFactory` boots the **real** ASP.NET Core pipeline. Your API's `TmsDbContext` needs a database — but the test host has no connection string. Swapping in `UseInMemoryDatabase` gives the pipeline a lightweight store without external infrastructure.

Step 2: Make Program Accessible to Test Project

Open `TmsApi.Api/TmsApi.Api.csproj` (in your API startup project) and add `InternalsVisibleTo` or expose `Program`:

```
<ItemGroup>
  <AssemblyAttribute
Include="System.Runtime.CompilerServices.InternalsVisibleTo">
    <_Parameter1>TmsApi.Tests</_Parameter1>
  </AssemblyAttribute>
</ItemGroup>
```

Alternatively, at the bottom of `Program.cs` in `TmsApi.Api`:

```
public partial class Program { }
```

Step 3: Create a Custom WebApplicationFactory

Create `CustomWebApplicationFactory.cs` in `TmsApi.Tests`:

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using TmsApi.Infrastructure.Persistence;
```

```
namespace TmsApi.Tests;

public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {

// 1. Supply required test configuration (JWT secret, etc.)
        builder.ConfigureAppConfiguration((context, config) =>
        {
            config.AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["Jwt:Key"] = "ThisIsASecretKeyForTestingPurposesOnly123456!",
                ["Jwt:Secret"] =
"ThisIsASecretKeyForTestingPurposesOnly123456!",
                ["Jwt:Issuer"] = "TmsTestIssuer",
                ["Jwt:Audience"] = "TmsTestAudience"
            });
        });

// 2. Remove production DbContext and register InMemory with isolated
internal provider
        builder.ConfigureServices(services =>
        {
            services.RemoveAll<DbContextOptions<TmsDbContext>>();
            services.RemoveAll<DbContextOptions>();
            services.RemoveAll<TmsDbContext>();

            var inMemoryProvider = new ServiceCollection()
                .AddEntityFrameworkInMemoryDatabase()
                .BuildServiceProvider();

            services.AddDbContext<TmsDbContext>(options =>
            {
                options.UseInMemoryDatabase("TmsTestDb");
                options.UseInternalServiceProvider(inMemoryProvider);
            });
        });
    }
}
```

What this does:

1. Injects in-memory configuration keys for `Jwt:Key` so authentication middleware doesn't throw `ArgumentNullException` when reading missing appsettings/user secrets.

2. Uses `.AddEntityFrameworkInMemoryDatabase()` with `.UseInternalServiceProvider()` to isolate EF Core's in-memory services from production database provider registrations (such as PostgreSQL/Npgsql).

Step 5: Write the API Integration Test

Create `CoursesApiTests.cs` in `TmsApi.Tests`:

```
using System.Net;
using System.Net.Http.Json;

namespace Tms.Tests;

public class CoursesApiTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly HttpClient _client;

    public CoursesApiTests(CustomWebApplicationFactory factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetCourses_ReturnsOkAndPagedJson()
    {
        // Act — pin the V2 URL (see Versioning callout below)
        var response = await
_client.GetAsync("/api/v2.0/courses?page=1&pageSize=10");

        // Assert — check HTTP status 200 OK
        response.EnsureSuccessStatusCode();

        // TMS API contract check: PagedResponse<T> with items array
        var page = await
response.Content.ReadFromJsonAsync<PagedCoursesJson>();
```

```
        Assert.NotNull(page?.Items);
    }

    [Fact]
    public async Task CreateCourse_InvalidCode_ReturnsValidationError()
    {
        // Act — post invalid payload (empty code) to the V2 controller
        var response = await _client.PostAsJsonAsync("/api/v2.0/courses", new
        {
            code = "",
            title = "Intro to TMS Security",
            maxCapacity = 30
        });

        // Assert — validation failure returns 400 Bad Request or 422
Unprocessable Entity
        Assert.True(
            response.StatusCode is HttpStatusCode.BadRequest or
HttpStatusCode.UnprocessableEntity);
    }

    private sealed class PagedCoursesJson
    {
        public List<CourseRowJson> Items { get; set; } = default!;
        public int TotalCount { get; set; }
    }

    private sealed class CourseRowJson
    {
        public int Id { get; set; }
        public string Code { get; set; } = "";
        public string Title { get; set; } = "";
        public int MaxCapacity { get; set; }
        public int EnrollmentCount { get; set; }
    }
}
```

Step 6: Run the Suite

```
dotnet test
```

Expected result: `CustomWebApplicationFactory` launches the API in memory with an in-memory database. Both integration tests pass: GET returns `200 OK` with a paged response, and POST returns `400 Bad Request` when `Code` is empty.

> **Versioning : target the URL, assert on the contract (M7 V1/V2).** The TMS API ships two controller trees, `TmsApi.Api.Controllers.V1` and `TmsApi.Api.Controllers.V2`. Their response envelopes differ: V1 returns a flat `{ "error": "..." }` JSON, V2 returns RFC 9457 `application/problem+json` with `type`, `title`, `detail`, and `extensions["code"]`. Two rules keep your integration tests from breaking every time someone bumps a version:

1. **Pin the URL.** Use `/api/v2.0/courses?page=1&pageSize=10` (not `/api/courses`) when the controller you care about lives in `V2`. Tests that hit the unversioned root are at the mercy of the route table.
2. **Assert on the stable `error.Code`, not the envelope shape.** The `EnrollmentError` sealed record (M7, `TmsApi.Application.Common`) carries a machine-readable `Code` (`"already_enrolled"`, `"course_full"`, `"course_not_found"`). The handler maps that code into whatever envelope the controller version returns. Your assertion stays stable:
   - ```
     // Stable across V1 (flat JSON) and V2 (ProblemDetails)
     var body = await response.Content.ReadFromJsonAsync<JsonElement>();
     Assert.Equal("already_enrolled",
     body.GetProperty("extensions").GetProperty("code").GetString()); // V2
     // — or for V1 —
     Assert.Equal("already_enrolled", body.GetProperty("code").GetString());
     ```
   - Don’t assert on the human `Message`, the `detail` text, or the `type` URI. Those are presentation, not contract.
     ```

Verification Checkpoint (Session 1)

- ☐ `TmsApi.Tests` project created and added to the solution, referencing `TmsApi.Application` and `TmsApi.Api`
- ☐ `TmsApi.Application/Grading/GradingService.cs` and `GradeLevel.cs` added
- ☐ At least one `[Fact]` test for a specific grade scenario (`CalculateLetterGrade(85, 100) → Distinction`)
- ☐ At least one `[Theory]` with boundary values (zero, distinction threshold, pass threshold, negative, exceeds-max, zero-max)
- ☐ NSubstitute installed and used to mock `IEnrollmentService`
- ☐ `Microsoft.AspNetCore.Mvc.Testing` installed
- ☐ `WebApplicationFactory<Program>` starts the API in memory
- ☐ GET test verifies `200 OK` and `items[]` array in `PagedResponse`
- ☐ POST test verifies validation failure (`400` or `422`) on invalid payloads
- ☐ `dotnet test` runs 100% green